

This course has already ended.

« Chapter 19.2: Own graphical user interfaces (http... Threads » (https://plus.cs.aalto.fi/studio_2/k2022/t...
CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 19 (https://plus.cs.aalto.fi/studio_2/k2022/w19/)
/ Chapter 19.3: Own graphical user interfaces, animations and threads

Chapter 19.3: Own graphical user interfaces, animations and threads

About this page:

Key questions How do I draw my own graphics into a window?
What about animation?

Topics Drawing graphics in the swing interface, the role of the
event listening thread in computation and animation.

What Will I Do? Ready-made examples, reading and answering a
few questions.

Indicative difficulty level: Easy

Rough Estimate of Workload:? 1 hour

Point Value: S10

Related *Projects:* Asteroids 
(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-
horrible-gitmanager-interface/projects/given/Asteroids/),
GUIThreads 
(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-
horrible-gitmanager-interface/projects/given/GUIThreads/)

Thread

Until now, the programs that have been built in this or the previous course are almost without exceptions single-threaded. In a multi-threaded program, several tasks can be performed in parallel. In practice, this rarely happens so that the tasks actually occur simultaneously in different processor cores. It is much more common that the execution of different threads is interleaved and the threads are given execution time one at a time. Switching execution times between threads happens so often that the end result seems to be parallel execution.

We will discuss threads more in their own separate chapter *Threads*.

In a single-threaded program, there is a *runtime stack* for performing methods and functions. The state of the objects is stored in another memory area, commonly referred to as *heap*. In a multi-threaded program, each thread has its own execution stack, but the objects are common. The use of these common parts from multiple threads at the same time can lead to surprising results, and that's why multiple structures are used to control concurrency. You can learn more about concurrency e.g. in later courses.

User interface and threads

In Swing, graphics and event handling are provided by one thread, the so-called **event dispatch thread**, also called *event listening* or *event processing thread*. When the user interface is drawn, this thread ultimately manages the graphics screen. When someone clicks the user interface with the mouse, this thread creates an event instance and conveys it to the event listening method. Changes to the user interface components etc., should be carried out in this thread. This avoids a number of possible problems, but it requires that some things need to be reported separately as tasks in this thread.

In one thread, there is quite a lot of processing power, and as long as handling of events and drawing the graphics takes place quickly, you manage without extra threads. However, when either of these takes a longer time, there will be problems, because the event processing thread will not be able to perform its other tasks normally.

You can investigate this issue in the project GUIThreads, whose code is also listed below. The given program calculates the Fibonacci numbers recursively, which requires quite a lot of memory and processing time when the computed fibonacci number approaches fifty.

In our example, there are two sliders to set how large Fibonacci number should be calculated. The upper controller performs the calculation in the event listening thread, the lower one performs it in a separate Future.

Try first sliding the lower slider close to the right end. The slider should turn gray during computation and finally turn back to normal, resulting in the calculation result which is shown below the slider. Move the slider to the right until counting takes some ten seconds.

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 7

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 10 / 10

If you move the upper controller to the same position and change the window size with the mouse, what happens?

- ☐ The window is updated normally (i.e. its contents are drawn).
- ☐ The program crashes.

☒ **The window is not updated normally.**

☐ The window size cannot be changed during the calculation.

✓ Correct!

Submit

Once you have seen the result, you recognize why threads are needed when computing takes more than a moment time. Try again to close the window during execution started from various slider settings.

```

import scala.swing._
import scala.swing.event._
import scala.collection.mutable.HashMap
import scala.concurrent.ExecutionContext.Implicits.global
import scala.concurrent.Future

object SlowFibonacci {

  // Most stupid and slowest way possible to calculate fibonacci numbers
  def calc (n: Int):Int =
    if (n == 0 || n == 1)
      n
    else
      calc(n - 1) + calc(n - 2)
}

```

```

object GUIThreadExample extends SimpleSwingApplication {

  val minFib    = 1
  val maxFib    = 50
  val startFib = 15

  def makeFibonacciCounter(fibName: String) = new Slider{
    orientation = Orientation.Horizontal
    min    = minFib
    max    = maxFib
    value  = startFib
    majorTickSpacing = 10
    paintTicks = true

    minimumSize = new Dimension(200, 300)

    val labelTable = HashMap[Int, Label]()
    labelTable += minFib    -> new Label(fibName)
    labelTable += maxFib/2  -> new Label("Fast")
    labelTable += 50        -> new Label("Quite long")
    labelTable += maxFib    -> new Label("Forever")

    labels = labelTable
    paintLabels = true
  }

  val eventThreadCounter = makeFibonacciCounter("Event thread")
  val otherThreadCounter = makeFibonacciCounter("Other thread")

  val numLabel = new Label(startFib.toString) {
    minimumSize = new Dimension(200,100)
  }

  def top = new MainFrame {
    contents = new BoxPanel(Orientation.Vertical) {
      contents += eventThreadCounter
      contents += otherThreadCounter
      contents += numLabel
    }

    size = new Dimension(700,200)
  }

  listenTo(eventThreadCounter)
  listenTo(otherThreadCounter)
}

```

```

listenTo(otherThreadCounter)

this.reactions += {
  case ValueChanged(source) =>
    val slider = source.asInstanceOf[Slider]

    if (!slider.adjusting) { // Calculate after the user stops moving the slider, not before

      slider.enabled = false // disable the slider (should go gray and unusable)

      if (slider == eventThreadCounter) {
        // Bad Idea - heavy calculation in the event dispatch thread

        val result = SlowFibonacci.calc(slider.value)
        numLabel.text = result.toString
        slider.enabled = true
      }
      else {
        // Better idea - Heavy calculation in a Future

        scala.concurrent.Future {
          SlowFibonacci.calc(slider.value)
        }.onSuccess {case result =>
          // Future runs in a different thread, so update the GUI
          // using the method Swing.onEDT which executes it's contents
          // in the event dispatch thread

          Swing.onEDT {
            numLabel.text = result.toString
            slider.enabled = true
          }
        }
      }
    }
  }
}

```

Animation

How can one produce simple animation in a swing program? Visit the Asteroids project and try to launch it. By clicking on asteroids, they split into smaller particles.

The calculation of this program is so light that it is performed in an event listening thread. If necessary, the same calculation could be transferred to a separate thread that would draw the image at the end of the computation, whereafter the event listening thread would only transfer it to the screen. In this example, however, the calculation has been considered deliberately light.

The animation is run by the `javax.swing.timer` class, which sends `ActionEvent` events to the event listener thread at regular intervals. In our example, the events are received by a "traditional" Java `ActionListener`. Each event causes `actionPerformed` call, which moves the asteroids in space one step further and asks the event listener thread to redraw the space-displaying component. It is indeed a request only, because the thread carries out drawing only when it gets execution time for it and just now it is carrying out the current `actionPerformed` method.

```
... code snippet from AsteroidGame ...

// This event listener and swing timer allow a periodic repetitive
// activity in the event listening thread. The game is light enough
// to be drawn in the thread without additional buffers or threads.

val listener = new ActionListener(){
  def actionPerformed(e : java.awt.event.ActionEvent) = {
    Space.step()
    arena.repaint()
  }
}

// Timer sends ActionEvent to ActionListener every 6ms,
// when the space moves forward and the screen is redrawn.
// This code therefore implements the animation

val timer = new javax.swing.Timer(6, listener)
timer.start()
```

Have a closer look at the code in the project and read comments there.

Where to get more information?

The Scala.swing components' interface differs partly from Java, but from the point of view of the basics and possibilities, Java's swing-demos are worth skimming through: Using Swing Components  (<http://docs.oracle.com/javase/tutorial/uiswing/components/index.html>)

More information on concurrency in swing is also available in swing tutorials: Concurrency in Swing  (<http://docs.oracle.com/javase/tutorial/uiswing/concurrency/>)

Find more information about 2D graphics in the same place: 2D-Graphics  (<http://docs.oracle.com/javase/tutorial/2d/index.html>)

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

Loading assignment...

« Chapter 19.2: Own graphical user interfaces ([http...](#)

Threads » (https://plus.cs.aalto.fi/studio_2/k2022/t...